ECOLE NATIONALE DES SCIENCES APPLIQUEES

Beni Mellal – Filiere IACS

Pr. BE ELBAGHAZAOUI

COMPTE RENDU

TP N°3

Architecture Distribuee

Docker Compose – Nginx – MongoDB – Redis – RabbitMQ

Etudiant : Achraf HAMOUDA
Module : Architecture Logicielle
Date : 30 Mars 2026
Encadrant : Pr. BE ELBAGHAZAOUI

Annee Universitaire 2025–2026

Contents

1 Objectifs **2**

2 Architecture du Systeme **2**

3 Implementation **2**
3.1 Docker Compose 2
3.2 Configuration Nginx – Load Balancer 3
3.3 Routes API Node.js 3

4 Tests avec Postman **4**
4.1 Test 1 – Creer une commande (POST /commandes) 4
4.2 Test 2 – 1er GET /commandes (source MongoDB) 6
4.3 Test 3 – 2eme GET (Round-robin Nginx confirme) 7
4.4 Test 4 – 3eme GET dans les 30s (cache Redis actif) 8
4.5 Recapitulatif des tests 8

5 Questions de compte-rendu **9**

6 Conclusion **10**

1. Objectifs

Ce TP vise a deployer une **architecture distribuee complete** avec 6 composants interconnectes via Docker Compose. Le flux principal implementé est :

`POST commande` → **MongoDB** → **Cache Redis (TTL 30s)** → **Notification RabbitMQ**

Nginx assure le **load balancing Round-robin** entre deux instances Node.js.

2. Architecture du Systeme

Composant	Technologie	Role
Load Balancer	Nginx :80	Round-robin entre Instance-1 (:3001) et Instance-2 (:3002)
Serveurs x2	Node.js/Express	Logique metier : validation, traitement, reponse JSON
Base de donnees	MongoDB	Stockage persistant des commandes
Cache	Redis TTL=30s	Cache du GET /commandes (evite les requetes repetitives)
Bus de messages	RabbitMQ	Notification asynchrone apres chaque commande

Flux de donnees :

- `Postman` → `Nginx:80` → `Node.js:3001 / :3002`
- `Node.js` → `MongoDB` (stockage persistant)
- `Node.js` → `Redis` (cache GET /commandes)
- `Node.js` → `RabbitMQ` → Consommateur → Notification console

3. Implementation

3.1 Docker Compose

Listing 1. docker-compose.yml – orchestration des 6 composants

```yaml
version: '3.8'
services:
  mongodb:
    image: mongo:6
    ports: ['27017:27017']
    volumes: [mongo_data:/data/db]

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']

```

```
  rabbitmq:
    image: rabbitmq:3-management
    ports: ['5672:5672', '15672:15672']
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin

  app1:
    build: ./app
    environment: {PORT: 3001, INSTANCE: 'Instance-1'}
    ports: ['3001:3001']
    depends_on: [mongodb, redis, rabbitmq]

  app2:
    build: ./app
    environment: {PORT: 3002, INSTANCE: 'Instance-2'}
    ports: ['3002:3002']
    depends_on: [mongodb, redis, rabbitmq]

  nginx:
    image: nginx:alpine
    ports: ['80:80']
    volumes: ['./nginx/nginx.conf:/etc/nginx/nginx.conf']
    depends_on: [app1, app2]

volumes:
  mongo_data:
```

3.2 Configuration Nginx – Load Balancer

Listing 2. nginx/nginx.conf – Round-robin entre 2 instances

```
events { worker_connections 1024; }
http {
  upstream backend {
    server app1:3001;   # Instance 1
    server app2:3002;   # Instance 2
  }
  server {
    listen 80;
    location / {
      proxy_pass http://backend;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
    }
  }
}
```

3.3 Routes API Node.js

Listing 3. routes/commandes.js – POST et GET avec cache Redis

```
// POST /commande : creer une commande
router.post('/', async (req, res) => {
  const commande = new Commande({ produit, quantite, client });
  await commande.save();                          // 1. Sauvegarde
      MongoDB
```

```
  await redis.del('toutes_commandes');            // 2. Invalider le cache
  publierCommande({ id: commande._id, produit, client }); // 3. RabbitMQ
  res.status(201).json({ message: 'Commande creee',
    commande, traitee_par: INSTANCE });
});

// GET /commandes : lister avec cache Redis
router.get('/', async (req, res) => {
  const cached = await redis.get('toutes_commandes');
  if (cached) {
    return res.json({ source: 'CACHE Redis',
      traitee_par: INSTANCE, commandes: JSON.parse(cached) });
  }
  const commandes = await Commande.find().sort({ date: -1 }); // MongoDB
  await redis.setex('toutes_commandes', 30, JSON.stringify(commandes));
  res.json({ source: 'MongoDB', traitee_par: INSTANCE, commandes });
});
```

4. Tests avec Postman

4.1 Test 1 – Creer une commande (POST /commandes)

```
POST http://localhost/commandes
Content-Type: application/json
{ "produit": "Laptop Dell XPS", "quantite": 2, "client": "Ahmed
   Benali" }
```

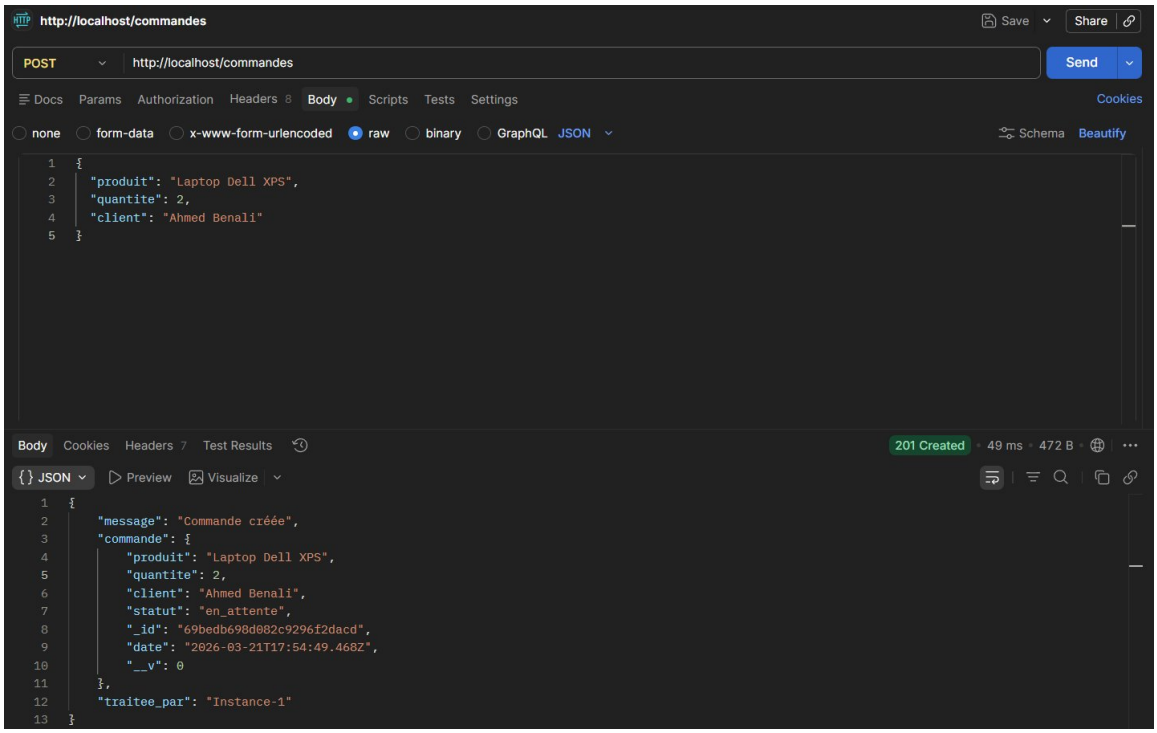

Figure 1. POST /commandes – 201 Created, commande stockee dans MongoDB

Resultat

`HTTP 201 Created` – Commande stockee dans MongoDB. Message RabbitMQ publie. `traitee_par: Instance-1`.

4.2 Test 2 – 1er GET /commandes (source MongoDB)

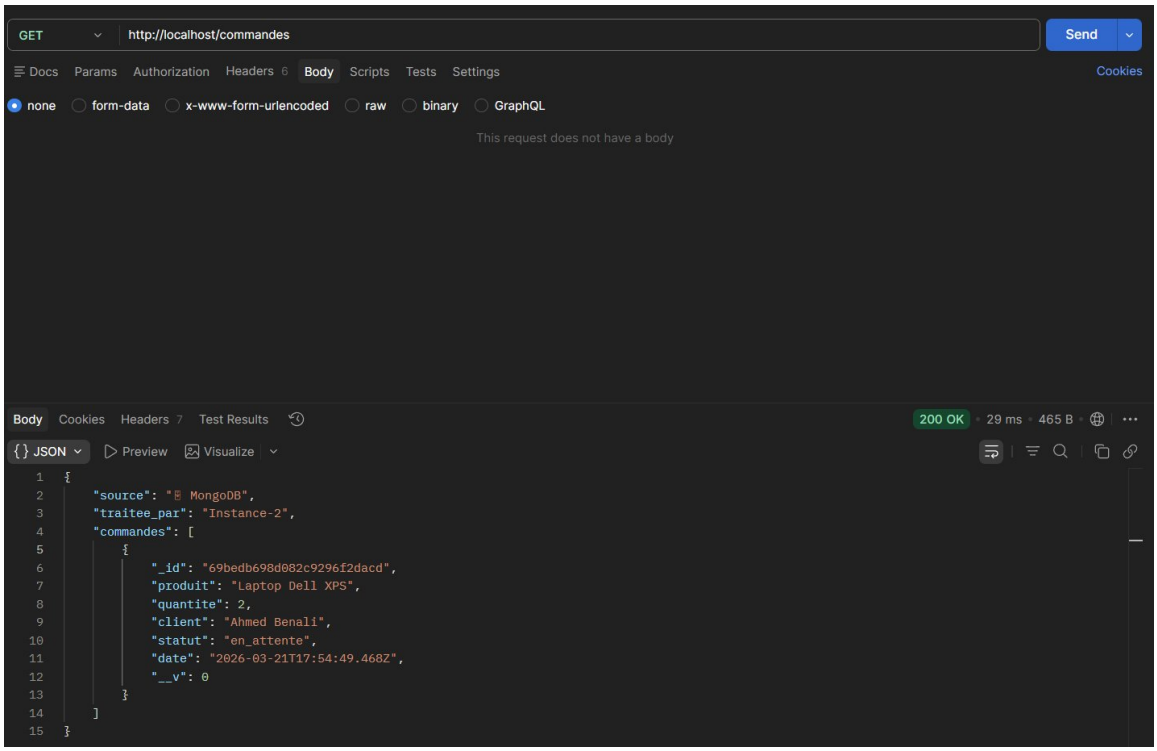

Figure 2. GET /commandes – 1er appel, source MongoDB, traitee par Instance-2

Resultat

`HTTP 200 OK` – `source: "MongoDB"` | `traitee_par: Instance-2` | Mise en cache Redis (TTL 30s).

4.3 Test 3 – 2eme GET (Round-robin Nginx confirme)

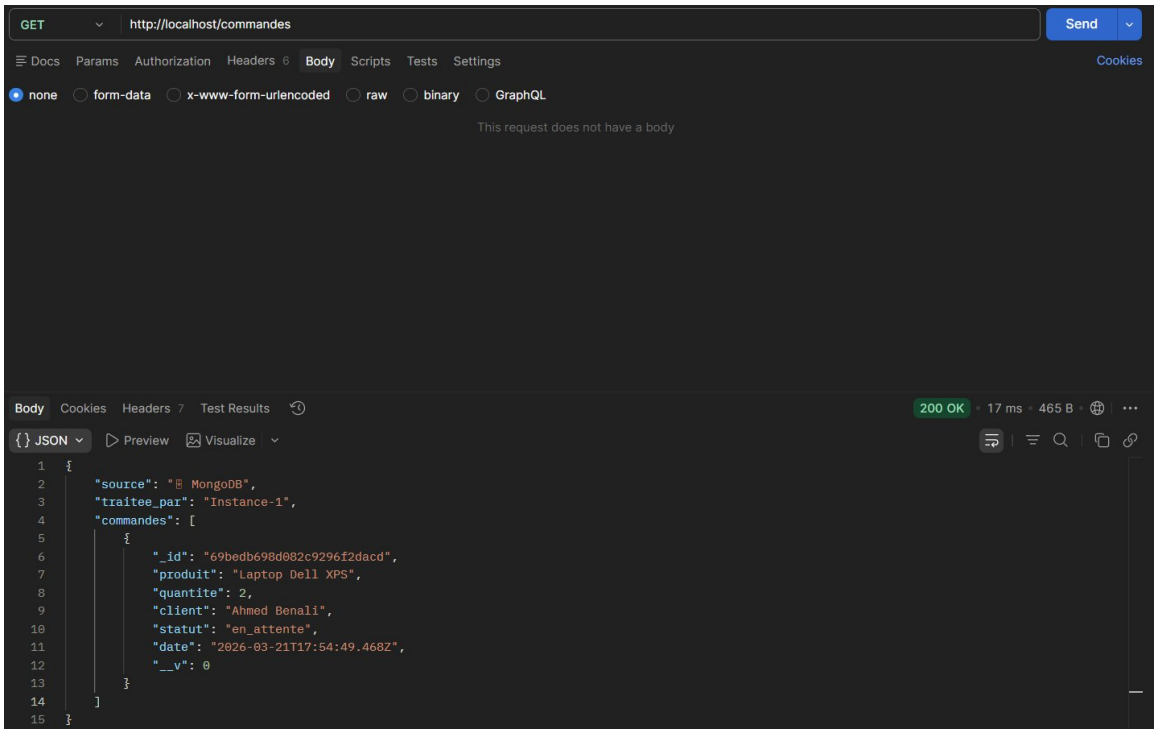

Figure 3. GET /commandes – 2eme appel, source MongoDB, traitee par Instance-1 (alternance confirmee)

Resultat

`HTTP 200 OK` – `source: "MongoDB"` | `traitee_par: Instance-1` | Alternance Instance-1/2 confirmee (Round-robin Nginx).

4.4 Test 4 – 3eme GET dans les 30s (cache Redis actif)

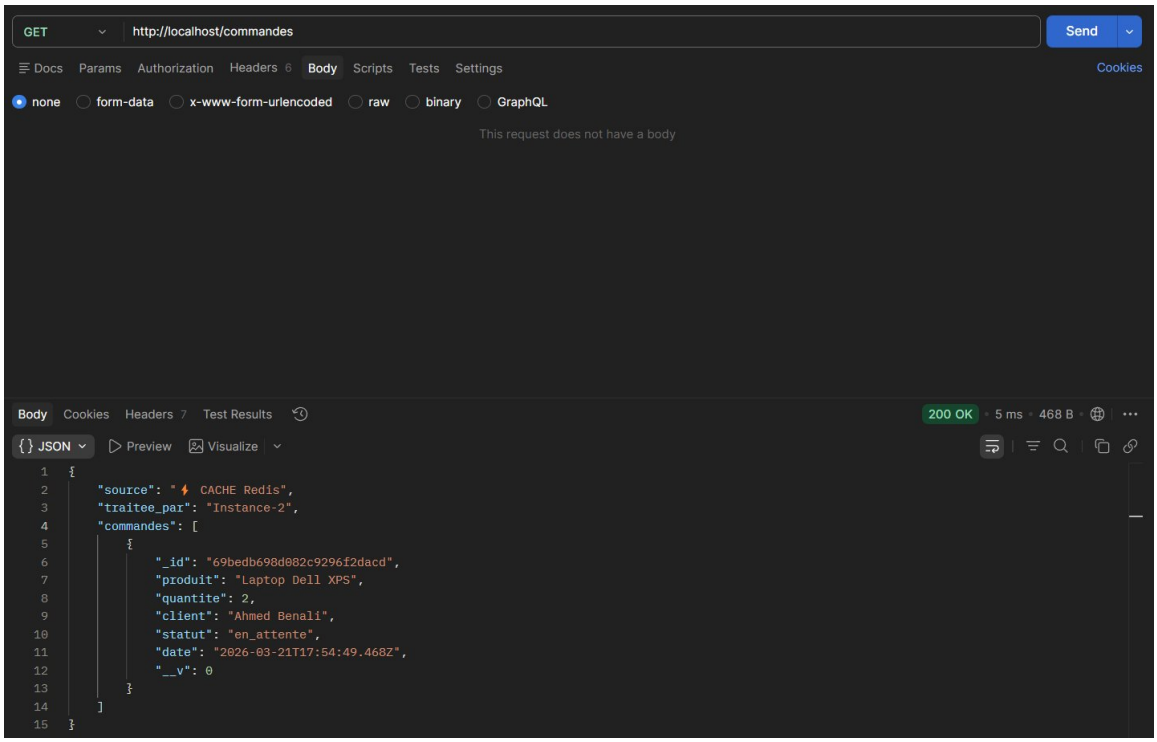

Figure 4. GET /commandes – 3eme appel dans les 30s, source CACHE Redis (5ms vs 17ms)

Resultat

`HTTP 200 OK` – `source: "CACHE Redis"` | **5ms vs 17ms** (MongoDB) | Cache Redis operationnel. La reponse est instantanee.

4.5 Recapitulatif des tests

Test	Methode	Source	Instance	OK
Creer commande	POST	MongoDB (stockage)	Instance-1	+
1er GET commandes	GET	MongoDB	Instance-2	+
2eme GET (Round-robin)	GET	MongoDB	Instance-1	+
3eme GET (cache actif)	GET	CACHE Redis (5ms)	Instance-2	+

5. Questions de compte-rendu

Q1 – Quel composant recoit en premier les requetes ? Quel algorithme utilise-t-il ?

Reponse : Le composant qui recoit en premier les requetes du client est **Nginx**, qui joue le role de **Load Balancer** sur le port 80. Il utilise l'algorithme **Round-robin** par defaut : les requetes sont distribuees en alternance entre les serveurs du pool. Ainsi, la requete 1 va vers `app1:3001`, la requete 2 vers `app2:3002`, la requete 3 vers `app1:3001`, etc. Cela garantit une repartition equitable de la charge entre les deux instances Node.js.

Q2 – Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il a fonctionner ?

Reponse : Oui, Nginx continue a fonctionner meme si `app1` tombe en panne. Nginx detecte automatiquement l'indisponibilite d'un serveur (apres quelques tentatives echouees) et redirige **toutes les requetes vers** `app2:3002` uniquement. Ce mecanisme s'appelle le **failover automatique**. L'application reste donc disponible pour les clients, ce qui est l'un des principaux avantages d'une architecture distribuee avec load balancer. Il suffirait d'ajouter `max_fails=3 fail_timeout=30s` dans la configuration Nginx pour un comportement encore plus robuste.

Q3 – Difference entre le 1er GET (source MongoDB) et le 2eme GET (source Redis)

Reponse : Lors du **1er GET**, aucune donnee n'est en cache : Node.js interroge **directement MongoDB**, recupere les commandes, puis les stocke dans Redis avec un TTL de 30 secondes. La reponse inclut `"source": "MongoDB"` et prend **17ms**. Lors du **2eme GET** (dans les 30 secondes), Node.js trouve les donnees dans **Redis** et les retourne immediatement sans solliciter MongoDB. La reponse inclut `"source": "CACHE Redis"` et prend seulement **5ms**. Le gain de performance est de **x3.4** car Redis opere en memoire vive, alors que MongoDB fait des operations disque.

Q4 – Pourquoi invalide-t-on le cache Redis apres un POST /commande ?

Reponse : Apres un `POST /commande`, une nouvelle commande est inseree dans MongoDB. Si on ne supprime pas le cache Redis, les prochains `GET /commandes` retourneraient encore l'ancienne liste (sans la nouvelle commande), ce qui provoquerait une **incoherence des donnees**. En executant `redis.del('toutes_commandes')`, on force le prochain GET a interroger MongoDB pour obtenir les donnees a jour, qui seront ensuite remises en cache. C'est le principe de **cache invalidation** : le cache doit etre invalide a chaque modification des donnees source.

Q5 – En quoi RabbitMQ ameliore-t-il la resilience du systeme ?

Reponse : RabbitMQ ameliore la resilience de plusieurs facons. Premierement, la communication est **asynchrone** : Node.js publie le message dans la file et continue sans attendre la reponse du consommateur. Si le service de notification est temporairement hors service, les messages sont **persistes dans la file** (option `durable: true`) et seront traites quand il reviendra. Deuxiemement, le **decoupage des responsabilites** signifie qu'une panne du consommateur n'affecte pas la creation des commandes. Enfin, le systeme peut facilement **scaler** en ajoutant plusieurs consommateurs qui traitent les messages en parallele.

Q6 – Comment ajouter une 3eme instance Node.js ?

Reponse : Pour ajouter une 3eme instance, il faudrait modifier **deux fichiers** :
1. Dans `docker-compose.yml` : ajouter un nouveau service `app3` :

```
app3:
  build: ./app
  environment: {PORT: 3003, INSTANCE: 'Instance-3'}
  ports: ['3003:3003']
  depends_on: [mongodb, redis, rabbitmq]
```

2. Dans `nginx/nginx.conf` : ajouter la ligne suivante dans le bloc `upstream backend` :

```
upstream backend {
  server app1:3001;
  server app2:3002;
  server app3:3003;  # Nouvelle instance
}
```

Nginx repartira alors les requetes en Round-robin entre les 3 instances de facon automatique.

6. Conclusion

Ce TP a permis de deployer une **architecture distribuee complete** avec 6 composants Docker. Les principaux concepts valides sont :

- **Load Balancing** : Nginx distribue les requetes en Round-robin entre deux instances Node.js, comme confirme par l'alternance `Instance-1`/`Instance-2` dans les tests.
- **Persistance** : MongoDB stocke les commandes de facon durable avec un schema Mongoose structure.
- **Cache** : Redis reduit le temps de reponse de 17ms a 5ms grace au cache avec TTL de 30 secondes.
- **Messagerie asynchrone** : RabbitMQ decouple la creation de commandes des notifications, ameliorant la resilience globale.

- **Containerisation** : Docker Compose orchestre l'ensemble des composants avec un seul fichier de configuration.